Intermediate Submission 1 & 2 Group 24

Ignacy Świderski, Tymon Szczepański, Floortje Lusthof, Amber Slob, Vincent van Roosmalen, Maciej Małek
Eindhoven University of Technology, The Netherlands
2272563, 2330113, 2316137, 2259656, 1679473, 2294192, Group 24

1 Intermediate Submission 1

1.1 Project Idea

This project focuses on developing an accessible audiobook player specifically for elderly users, addressing the common failure point of complex communication and poorly defined user needs. Our system prioritizes usability through an interface featuring clearly labeled buttons and a streamlined interface. The system provides essential playback functionalities, including the ability to play, pause, and resume audio files while automatically saving listening progress.

1.2 Use-Case Diagram

Use case: Playing audiobooks.

(1) The user views the overview of available audiobooks (FR-01).
(2) The user selects a specific audiobook and sees the episodes (FR-02).
(3) The system loads the file and begins playback (FR-04, FR-06).

Alternative scenario:
The system has additional playback functions to account for:

- The user adjusts the volume of the audiobook (FR-09).
- The user skips to the next/previous audio file in a series (FR-08).
- The user sets a sleep timer for the audiobook (FR-12).
- The user adjusts the playback speed (FR-13).

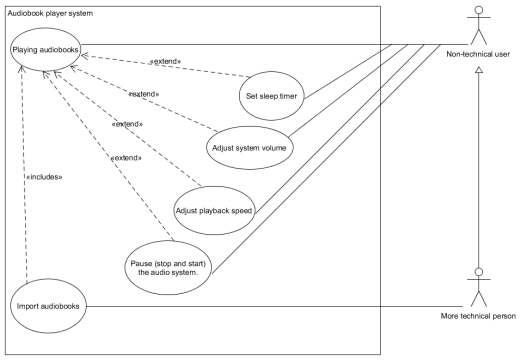


Figure 1: Use-Case Diagram

1.3 Use-Case Description

Section	Content
Identifier	UC-Audio-001
Name	Playing audiobooks
Authors	Ignacy Świderski, Tymon Szczepański, Floortje Lusthof, Amber Slob, Vincent van Roosmalen, Maciej Małek
Priority	Must
Summary	A user browses their library, selects an audiobook and episode, and listens to the audio while progress is tracked
Actors	User (Elderly person)
Trigger	User selects an audiobook to play
Pre-conditions	Audio files are available in the system (FR-04/FR-05)
Post-conditions	The system has saved the listening progress for the episode (FR-03)
Result	Audio output of the selected audiobook episode.
Main Scenario	The primary scenario: 1. The system displays an overview of available audiobooks (FR-01). 2. The user selects an audiobook. 3. The system shows an overview of all episodes (FR-02). 4. The user selects a specific episode. 5. The system begins audio playback (FR-06).
AlternativeScenario	User interacts with playback settings: 1. The user chooses to adjust volume (FR-09), skip files (FR-08), set a sleep timer (FR-12), or change playback speed (FR-13/FR-14). 2. The system applies the change and continues the main playback flow.
Exception Scenario	The selected file fails to load correctly: 1. The system identifies that the audio file is unreadable. 2. The system displays a clear error message to the user. 3. The system returns to the audiobook overview screen (FR-01).

Table 1: Use-Case Description

This table describes the main use-case of our program. In the table, all information about it can be found, including the description, description of an alternative and exception scenario, actors, triggers and pre- and post-conditions.

1.4 Requirements

Label	Type	Description	Priority
FR-01	Functional	The program must show users an overview of their available audiobooks on the home page.	Must
FR-02	Functional	The program must show users an overview of all episodes if they select an audiobook.	Must
FR-03	Functional	The program must save listening progress per episode.	Must
FR-04	Functional	The program must automatically load audio files from a prespecified default folder if the user puts any in it.	Must
FR-05	Functional	The program must provide a button to manually browse and open audio files from other locations for easy access.	Must
FR-06	Functional	The program must have a play/pause button to start or stop playing audio.	Must
QR-01	Quality	The program must be able to work with at least 100 audio files.	Must
CR-01	Restriction	The program must run on the Windows 11 operating system.	Must
FR-07	Functional	The program should be able to search for audio files in the device memory based on file name.	Should
FR-08	Functional	The program should be able to skip audio files, more specifically audio files in a series of audio files.	Should
FR-09	Functional	The program should be able to adjust the volume of the audio being played.	Should
FR-10	Functional	The program should be able to play large audiofiles extending a one hour playing time.	Should
QR-02	Quality	The program should be able to start playing an audiofile within 2 seconds from being selected.	Should
FR-11	Functional	The program could be able to search for audio files, based on meta data.	Could
FR-12	Functional	The program could offer an automatic pause timer, i.e. sleep timer.	Could
FR-13	Functional	The program could adjust the playback speed of the audio being played.	Could
FR-14	Functional	The program could play audio twice as fast and half as slow.	Could
FR-15	Functional	The program won't have access to the internet.	Won't

Table 2: Requirements

This table lists all the requirements. For every requirement, its label, type, description and priority can be found.

2 Intermediate Submission 2

2.1 Activity Diagram

Activity diagram description: Initially, the program is started and the save data is loaded (program state request).
There are 2 loops that can be identified:
1) The main program loop: Determining the available audiobooks, selecting an audiobook, playing an audiobook untill it finishes. This loop can be restarted by importing an audiobook, pausing an audiobook, or when the sleep timer is triggered. This loop contains an alternative path for resetting the sleep timer. The loop is exited when the program is shut down.
2) A subloop: Playing an audiobook. This loop contains an alternative path for audiobook replay adjustments. When there is an audio playing error (due to a corrupted file, etc.) the loop is exited and an error message will be displayed.

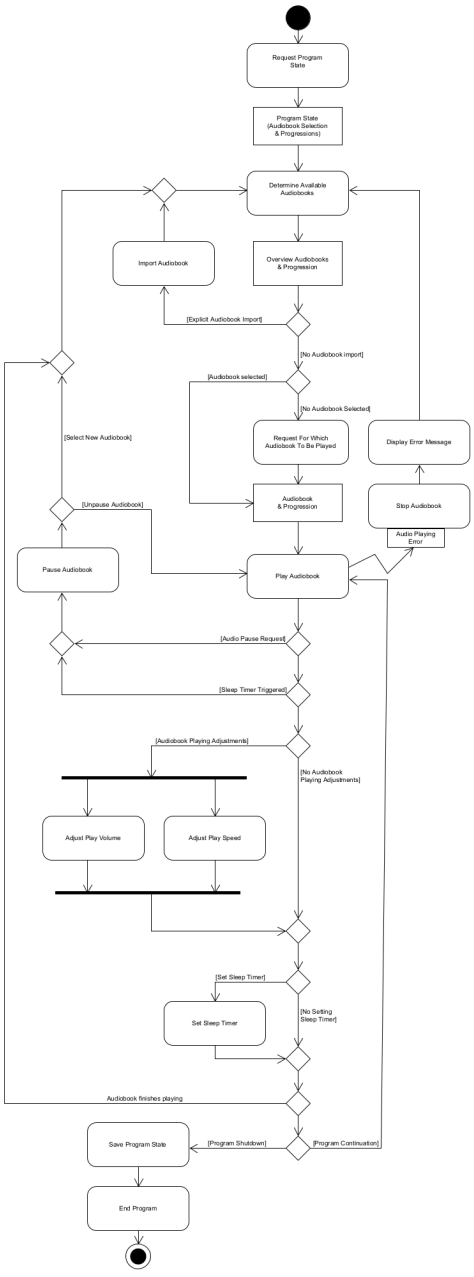


Figure 2: Activity diagram

2.2 Testing Strategy

⇒ What do we test

The testing strategy targets the main use case UC-Audio-001 and all requirements linked to it. Concretely, the following behaviors are tested:

- **Core playback flow** (FR-01, FR-02, FR-04, FR-06): the system correctly loads audio files from the default folder, displays the library and episode list, and starts playback on selection.
- **Progress saving** (FR-03): after playing a portion of an episode, the saved progress matches the position at which playback was paused or stopped.
- **Manual file import** (FR-05, FR-07): browsing and opening a file from a non-default location results in the file appearing and being playable.
- **Playback controls** (FR-06, FR-08, FR-09, FR-13/FR-14): play/pause, skip to next/previous episode, volume adjustment, and playback speed changes all behave as specified.
- **Sleep timer** (FR-12): playback stops at the configured time.
- **Performance** (QR-01, QR-02, FR-10): the system handles a library of at least 100 audio files without degradation, and playback begins within 2 seconds of file selection and the system is able to play large audiofiles extending a one hour playing time..
- **Exception scenario**: when a selected audio file is unreadable, the system displays a clear error message and returns to the library overview (FR-01), without crashing.

⇒ How do we test

We use black-box unit testing as the primary approach, deriving test cases directly from the requirements and the use-case specification rather than from internal implementation details. This ensures we verify the behavior the user actually experiences. Where internal state needs to be verified (e.g. confirming that the saved progress value is numerically correct), we supplement with white-box tests that inspect internal variables.

Tests are implemented using JUnit 5 and follow a test-driven development (TDD) workflow: test cases for each requirement are written before or alongside the corresponding implementation, so that failing tests act as the development target.

Test cases are structured around the following strategies:

- **Equivalence classes:** for playback speed, we test one representative value in each class (below minimum, at minimum, normal range, at maximum, above maximum). Similarly for volume.
- **Boundary values:** we test edge cases such as an empty library (0 files), exactly 1 file, exactly 100 files, and a file exactly at 1 hour in length (FR-10), as well as seeking to position 0 and to the very end of an episode.
- **Exception testing:** each `@throws` condition in the relevant methods is covered by a dedicated test that verifies the correct exception type is thrown and that its message is non-empty and meaningful. For instance, loading a corrupted or missing file must throw the appropriate exception and trigger the error-display behavior specified in the exception scenario.
- **Consistency:** multiple different audio files that are all valid should each trigger the same correct loading and playback behavior, confirming the implementation does not rely on file-specific quirks.

3 Intermediate Submission 3

Contributors: Ignacy Świderski, Maciej Małek, Tymon Szczepański, Floortje Lusthof, Amber Slob, Vincent van Roosmalen

3.1 Changes to Use Case and Requirements

The core use case has remained stable: a desktop audiobook player that allows a user to load audio files from a local folder, play them back with full transport controls, and resume from the last saved position across sessions.

One requirement was refined since the previous submission:

The sleep-timer requirement was made more precise - the timer must support cancellation at any point, and the remaining time must be displayed live in the UI.

No requirements were removed.

3.2 Project Structure

The project is a single Maven module under `com.mycompany.irr00_group_project`. All production classes reside in `src/main/java` and all tests in `src/test/java`.

- **Model layer (merged to `main`):**
  - `Episode`: Represents one audio file (title, path, duration, progress).
  - `Audiobook`: An ordered list of episodes with a title.
  - `Library`: Scans `~/Audiobooks` and builds audiobooks from subdirectories, treating each audio file (.mp3, .wav, .m4a) as an episode.
  - `ProgressStore`: Persists the playback position of every episode to a `.properties` file in `~/.audiobookplayer/`.
  - `SleepTimer`: Fires a callback after a configurable delay and supports cancellation.
- **Playback layer:**
  - `PlaybackController`: An interface defining play, pause, stop, seek, volume, and speed.
  - `AudioPlayer`: Implements the controller using the JavaFX `MediaPlayer` API.
- **UI layer (in feature branches, integration in progress):**
  - `PlayerViewController` + `PlayerView.fxml`: Provide the main playback screen with transport buttons, a seek slider, volume and speed controls, and the sleep timer widget.
  - `SettingsController` + `settings.fxml`: Provide a settings screen with theme selection; `Settings` and `Theme` model the global application preferences, persisted via `Constants.SETTINGS_FILE`.
  - `AudiobookOverview`: Displays the full library and lets the user select a book to play.
  - `EpisodeViewerController` + `EpisodeView.fxml`: Displays the episodes within an audio book and lets the user select an episode to play.
  - `HomeView` + `HomeViewController` + `homeview.fxml`: Provide the welcome screen, with an overview of recently played audiobooks and buttons to go to settings or library.

`App` is the JavaFX entry point that wires these components together. Every class has a corresponding JUnit 5 test class.

3.3 Project Status

We are on schedule and have largely fulfilled our use case.

The full model and playback engine are complete and tested (merged to `main`). The player UI is feature-complete on the `player-view` branch and awaits integration review.

The settings screen and the library browser are developed on separate branches and are ready for review and merging.

The remaining work is integration: merging the three feature branches, connecting the library browser to the player screen, and end-to-end manual testing. We do not expect to need to drop any planned requirement before the final deadline.